Windows Service & Process Monitoring Agent

Real-time Windows defender-style agent that watches processes and services for suspicious parent-child chains, rogue startup entries, and unauthorized executables. Designed for blue-team labs, incident-response drills, or classroom demos.

Features

- Parent-child anomaly detection (e.g., winword.exe -> powershell.exe)
- Startup service audit to flag new or misconfigured services
- Whitelist/blacklist-driven unauthorized process detection, with path heuristics (Temp/AppData)
- Structured alert log plus human-readable report (JSON + TXT)
- Graceful fallbacks when WMI/pywin32 are not installed (process monitoring still works)

Quick Start (PowerShell)

1. python -m venv .venv
2. ./.venv/Scripts/Activate.ps1
3. pip install -r requirements.txt
4. Run a single sweep:
   python -m src.monitor --once
5. Run continuously (default 10s interval):
   python -m src.monitor --interval 15

Artifacts land in:

- logs/detections.log → rolling log of alerts
- reports/ → timestamped JSON and TXT summaries per sweep

How Detection Works

- **Parent-Child rules:** configurable pairs live in src/config.py under SUSPICIOUS_PARENT_CHILD. Any matching chain emits a high-severity alert.
- **Unauthorized processes:** anything not on the whitelist or explicitly blacklisted, running from user-writable locations (Temp/AppData) is flagged.

- **Startup services:** WMI service enumeration looks for new or odd file paths and services set to auto-start from non-system locations.

Customizing

- Edit src/config.py to tweak whitelists, blacklists, and rules
- Provide external lists via --whitelist / --blacklist (one process name per line)
- Adjust sweep cadence with --interval or use --once for a snapshot

Reporting

Every sweep writes two files in reports/:

- report-YYYYMMDD-HHMMSS.json → machine-friendly alerts and metadata
- report-YYYYMMDD-HHMMSS.txt → concise human-readable summary

Limitations

- Requires Windows with PowerShell and Python 3.9+
- Service auditing uses WMI; if wmi/pywin32 are not available, the script skips that step but continues process monitoring

Project Structure

- src/config.py → detection rules and defaults
- src/monitor.py → main agent logic and CLI
- src/reporting.py → logging and report writers
- requirements.txt → Python dependencies

Next Steps / Ideas

- Add signature verification (Authenticode) to reduce false positives
- Emit Windows Event Log records for SIEM ingestion
- Add YARA-based module scanning or ETW consumer for richer telemetry

Technical Documentation – Windows Service & Process Monitoring Agent

Project Overview / Description

Agent monitors Windows processes and services to surface malicious, unauthorized, or suspicious behavior. It inspects parent-child process chains, startup services, and process locations to flag anomalies and produces human-readable and JSON reports.

Practical Motivation

- Windows compromise paths often rely on malicious services, abnormal parent-child chains, persistence via startup entries, or fake executables in user-writable paths
- Early detection reduces risk of malware execution, privilege escalation, and stealth persistence

Project Objectives

- Monitor active processes and evaluate parent-child behavior
- Audit startup services for suspicious or modified entries
- Detect unauthorized or unknown processes using whitelist/blacklist logic
- Generate alerts from rule-based triggers
- Produce detailed reports of anomalies

Practical Scope of the Project

Parent-Child Relationship Monitor

- Tracks PID and PPID
- Compares against detection rules

Startup Service Audit

- Enumerates services via WMI
- Flags unusual paths or auto-start behavior
- Detects new or modified services

Unauthorized Process Detection

- Uses whitelist/blacklist
- Flags unknown or unsigned processes

- Focus on user-writable directories (Temp/AppData/Public/Downloads)

Reporting & Alert System

- Logs alerts to logs/detections.log
- Generates JSON + TXT reports

Tools & Technologies Used

- Python 3.9+
- Modules: psutil, wmi, pywin32, colorama
- PowerShell (optional)
- Markdown / documentation tools

Practical Techniques Implemented

Detection Techniques

- Parent-child behavior monitoring
- Service configuration auditing
- Whitelist/blacklist detection
- Path-based persistence detection

Blue Team Techniques

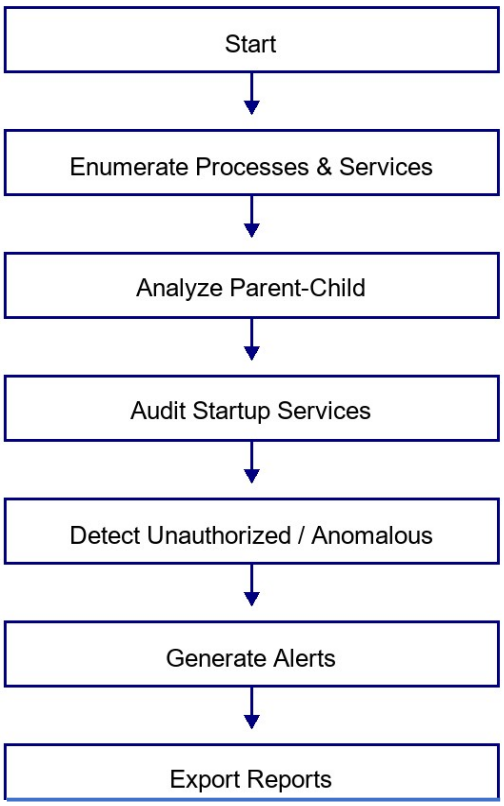
- Real-time monitoring
- Suspicious activity detection
- Service anomaly identification
- Security baseline reinforcement

Workflow / Architecture

1. Enumerate active processes
2. Analyze parent-child chains
3. Audit startup services
4. Detect unauthorized processes
5. Generate alerts

6. Export reports

Flowchart



Expected Practical Output

- Parent-child anomalies
- Suspicious startup services
- Unauthorized processes
- Timestamped logs
- Final reports

Learning Outcomes

- Windows process architecture understanding
- Malware behavior knowledge
- Real-time monitoring techniques
- Detection logic design

- Defensive security engineering

Project Deliverables

- Monitoring agent toolkit
- Baseline file
- Logs and reports
- Documentation
- Flowchart
- Detection rules

How to Run and Validate

- Single sweep:
  python -m src.monitor --once
- Continuous monitoring:
  python -m src.monitor --interval 15
- Service drift detection:
  --use-baseline --baseline-file state/service_baseline.json

Current Compliance Status

- Parent-child detection: Implemented
- Startup service audit: Implemented
- Unauthorized detection: Implemented
- Reporting: Implemented
- Documentation: Provided